Naviguer dans l'historique

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. Présentation de git log **3**

A. Afficher l'historique 3

B. Filtrer l'historique....6

C. Exercice : Quiz....6

III. Interaction avec la sortie de git log **7**

A. Interaction avec la sortie de git log 7

B. Exercice : Quiz....8

IV. Essentiel **9**

V. Auto-évaluation **9**

A. Exercice 9

B. Test....11

Solutions des exercices **11**

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée : 1 heure

Environnement de travail : GitBash / Terminal

Prérequis : avoir installé Git sur sa machine

Contexte

Cela vous est déjà sûrement arrivé, vous retrouvez un morceau de code dans votre projet, mais vous ne savez plus pourquoi vous l'avez écrit. Ou alors vous tombez sur du code produit par quelqu'un d'autre, mais vous ne le comprenez pas. Vous aimeriez alors avoir plus d'informations sur le contexte de ce code, pourquoi et à quel moment il a été fait. Git répond parfaitement à cette demande, grâce à son historique puissant.

Dans le développement web, l'utilisation de Git et des commits est essentielle pour enregistrer les modifications apportées à un projet. Un commit représente une version de votre projet à un moment précis. Le tout accompagné d'un message explicatif. Ces commits s'accumulent au fil du temps, formant un historique détaillé des changements effectués. Pour afficher cet historique, la commande « `git log` » est utilisée, fournissant une liste chronologique des commits avec des informations telles que l'identifiant, l'auteur, la date et le message du commit.

Parfois, il est nécessaire d'examiner les modifications spécifiques effectuées dans un commit, ce qui peut être réalisé en utilisant l'option « *-p* » pour afficher les détails des modifications. Cependant, lorsque l'historique est trop long, il peut être difficile de naviguer dans l'affichage de git log, mais en appuyant sur la touche « *q* », on peut revenir à l'invite de commande. De plus, il est possible de filtrer l'historique en affichant uniquement un certain nombre de commits les plus récents en utilisant l'option « *-x* » avec git log.

II. Présentation de git log

A. Afficher l'historique

Les commits

Un commit, dans le contexte de Git et du développement web, est une action qui enregistre une ou plusieurs modifications dans un dépôt Git. Lorsque vous effectuez un commit, vous créez un instantané (ou une version) des fichiers du projet à un moment précis, en enregistrant les changements qui ont été apportés depuis le dernier commit, en y joignant un message de commit, pour expliquer les changements.

Au fur et à mesure du développement, nous enchaînons donc les commits, et en finalité, nous avons un historique de commits, permettant de retrouver chaque changement dans le code. Nous comprenons donc que plus nos commits sont réguliers, et plus le message de commit est clair et précis, plus l'historique sera utilisable.

Méthode **Afficher l'historique des commits**

Lorsque vous exécutez la commande « *git log* » dans un dépôt, vous obtenez une liste des commits effectués, triés par ordre chronologique.

```
1 git log
```

Nous aurons un retour similaire à celui-ci :

```
1 commit ca82a6dff817ec66f44342007202690a93763949
2 Author: Scott Chacon <schacon@gee-mail.com>
3 Date:   Mon Mar 17 21:52:11 2008 -0700
4
5     Change version number
6
7 commit 085bb3bcb608e1e8451d4b2432f8ecbe6306e7e7
```

```
Author: Scott Chacon <schacon@gee-mail.com>
Date:   Sat Mar 15 16:40:33 2008 -0700

    Remove unnecessary test

commit a11bef06a3f659402fe7563abf99ad00de2209e6
Author: Scott Chacon <schacon@gee-mail.com>
Date:   Sat Mar 15 10:31:28 2008 -0700

    Initial commit
```

Nous pouvons voir quatre informations différentes par commit :

- Identifiant du commit : il s'agit d'une chaîne de caractères unique qui identifie de manière unique un commit. Cet identifiant est souvent représenté sous forme d'une longue chaîne hexadécimale.
- Auteur : l'auteur du commit est celui qui a fait le commit, son nom et son mail sont paramétrés dans les configurations de git.
- Date du commit : très utile pour savoir quand la modification a été faite (par exemple, nous retrouvons un bug depuis le 17/03, et nous nous rendons compte qu'il y a eu un commit ce jour-là. Le bug est donc sûrement dans la modification de ce commit.).
- Message du commit : chaque commit est accompagné d'un message qui décrit les modifications effectuées. Le message du commit doit être informatif et décrire clairement les changements apportés.

Message de commit

En soi, mettre un message de commit n'est pas une obligation technique, vous pourriez utiliser cette commande pour faire un commit sans message.

```
git commit -a --allow-empty-message -m ''
```

Seulement, c'est une très mauvaise pratique, il vaut mieux toujours mettre un message, car il n'y a pas de cas où nous faisons un commit sans raison, et le message est là pour spécifier la raison du commit.

Remarque **Affichage trop long**

Si vous avez beaucoup de commit, vous pourriez vous sentir coincé dans l'écran de git log, car le terminal ne répond plus comme avant. Pour sortir de l'écran de git log et revenir à l'invite de commande, vous pouvez appuyer sur la touche q (pour « *quitter* ») de votre clavier. Cela permet de quitter l'affichage du log et de revenir à votre terminal ou à l'interface de ligne de commande que vous utilisez.

Lorsque vous appuyez sur la touche q, Git annule l'affichage de git log et vous ramène à l'endroit où vous avez exécuté la commande. Vous pouvez alors continuer à utiliser Git ou exécuter d'autres commandes selon vos besoins.

Méthode **Afficher l'historique et le détail de toutes les modifications**

Si vous êtes à la recherche d'un bug, vous pourriez vouloir voir quelles sont les modifications qui ont été faites pour chaque commit. Pour cela, vous pouvez utiliser l'option -p ou --patch

```
git log -p
```

Pour un seul commit, nous aurions ce résultat

```
commit ca82a6dff817ec66f44342007202690a93763949
Author: Scott Chacon <schacon@gee-mail.com>
Date:   Mon Mar 17 21:52:11 2008 -0700

    Change version number
```


Tous droits réservés à STUDI - Reproduction interdite

```
diff --git a/Rakefile b/Rakefile
index a874b73..8f94139 100644
--- a/Rakefile
+++ b/Rakefile
@@ -5,7 +5,7 @@ require 'rake/gempackagetask'
 spec = Gem::Specification.new do |s|
     s.platform  =   Gem::Platform::RUBY
     s.name      =   "simplegit"
-    s.version   =   "0.1.0"
+    s.version   =   "0.1.1"
     s.author    =   "Scott Chacon"
     s.email     =   "schacon@gee-mail.com"
     s.summary   =   "A simple gem for using Git in Ruby code."
```

Comment comprendre tout cela ?

Nous avons ici un commit spécifique avec l'identifiant ca82a6dff817ec66f44342007202690a93763949. L'auteur du commit est Scott Chacon, dont l'adresse e-mail est schacon@gee-mail.com[1]. La date du commit est le lundi 17 mars 2008 à 21 h 52 (fuseau horaire -0700).

Ensuite, nous avons la partie diff, qui montre les différences apportées au fichier Rakefile :

`index a874b73..8f94139 100644` : ces informations indiquent les identifiants des versions précédente et actuelle du fichier Rakefile, ainsi que les droits d'accès (100644).

`--- a/Rakefile` : indique la version précédente du fichier Rakefile.

`+++ b/Rakefile` : indique la version actuelle du fichier Rakefile.

Ensuite, nous avons les lignes de code modifiées :

La ligne `s.version = "0.1.0"` a été modifiée en `s.version = "0.1.1"`. Cela indique que la version du projet a été mise à jour de 0.1.0 à 0.1.1.

Le reste des lignes du fichier Rakefile reste inchangé dans ce commit.

Ce commit représente donc un changement de numéro de version (0.1.0 à 0.1.1) effectué par Scott Chacon dans le fichier Rakefile du projet.

Remarque **L'affichage des différences n'est pas très lisible ?**

Il est vrai qu'à première vue, l'affichage des différences entre deux versions de fichier peut paraître complexe. Pas de panique ! C'est une question d'habitude. Beaucoup de développeurs utilisent aussi des interfaces graphiques permettant d'avoir les informations remontées par Git de façon bien plus visuelle comme par exemple le logiciel gitKraken[2].

GitKraken vous permettra de voir la liste des commits (git log) et lorsque vous cliquerez sur le commit, vous aurez accès aux documents modifiés (git log -p). Puis en cliquant sur le fichier, vous accédez à une vue des modifications du document (ligne supprimée en rouge et ajoutée en vert). Pratique !

1 mailto:schacon@gee-mail.com
2 https://www.gitkraken.com/

Tous droits réservés à STUDI - Reproduction interdite

B. Filtrer l'historique

Méthode **Afficher les x derniers commits**

Sur certaines applications, nous pouvons arriver à plusieurs milliers de commits, donc nous voulons souvent vérifier uniquement les derniers. Git log nous permet de récupérer les x derniers commits avec l'option -x (pour afficher x commits).

Elle s'utilise ainsi :

```
#Afficher 1 commit
git log -1
#Afficher 2 commits
git log -2
#Afficher 3 commits
git log -3
#Afficher 4 commits avec le détail des modifications
git log -p -4
```

C. Exercice : Quiz

[solution n°1 p.13]

Question 1

Qu'est-ce qu'un commit dans Git ?

- O Un instantané des fichiers du projet à un moment précis
- O Une version des fichiers du projet incluant les changements depuis le dernier commit

Question 2

Quelle commande permet d'afficher l'historique des commits dans un dépôt Git ?

- O git log
- O git history
- O git show

Question 3

Quelles informations sont affichées pour chaque commit dans git log ?

- O Identifiant du commit, auteur, date du commit, message du commit
- O Numéro de version, date du commit, auteur, message du commit
- O Identifiant du commit, date du commit, message du commit, fichiers modifiés

Question 4

Comment quitter l'affichage de git log et revenir à l'invite de commande ?

- O Appuyer sur la touche x
- O Appuyer sur la touche q
- O Appuyer sur la touche Ctrl + C

Question 5

Tous droits réservés à STUDI - Reproduction interdite

Comment afficher les x derniers commits dans « *git log* » ?

- ○ git log -n x
- ○ git log --last=x
- ○ git log -x

III. Interaction avec la sortie de git log

A. Interaction avec la sortie de git log

Filtrer l'historique

Lorsque vous utilisez la commande `git log`, vous pouvez appliquer des options de filtrage pour affiner les résultats et n'afficher que les commits qui répondent à certains critères. Les options de filtrage vous permettent de sélectionner les commits en fonction de l'auteur, de la période ou de la branche.

Voici quelques-uns des filtres couramment utilisés avec la commande `git log` :

- Filtre par auteur : vous pouvez filtrer les commits pour n'afficher que ceux réalisés par un auteur spécifique. Pour ce faire, vous pouvez utiliser l'option --author=<nom> ou son raccourci -a. Par exemple, pour afficher les commits réalisés par John Doe, vous pouvez exécuter la commande : `git log --author="John Doe"`.
- Filtre par période : il est possible de filtrer les commits pour n'afficher que ceux effectués pendant une période spécifique. Vous pouvez utiliser l'option --since=<date> pour spécifier la date de début et l'option --until=<date> pour spécifier la date de fin. Par exemple, pour afficher les commits réalisés entre le 1er janvier 2023 et le 31 janvier 2023, vous pouvez exécuter la commande : `git log --since="2023-01-01" --until="2023-01-31"`.
- Filtre par branche : vous pouvez filtrer les commits pour n'afficher que ceux d'une branche spécifique. Il suffit d'indiquer le nom de la branche après la commande git log. Par exemple, pour afficher les commits de la branche « *develop* », vous pouvez exécuter la commande : `git log develop`.

Méthode **Afficher les détails d'un commit spécifique**

La commande `git show` est utilisée pour afficher les détails d'un commit spécifique dans le dépôt Git. La syntaxe de base est la suivante : `git show <identifiant du commit>`. L'identifiant du commit peut être le nom d'une branche, un identifiant court ou complet, ou un tag.

Afficher les détails d'un commit spécifique :

```
1 git show abcdefg
```

Cette commande affiche les détails du commit identifié par « *abcdefg* ».

Afficher les modifications apportées par un commit :

```
1 git show abcdefg --stat
```

En ajoutant l'option --stat, vous pouvez obtenir des statistiques sur les fichiers modifiés dans le commit, y compris les ajouts et les suppressions de lignes.

Git checkout

La commande git checkout permet de passer à un commit spécifique dans l'historique. Vous pouvez utiliser la syntaxe suivante : `git checkout <identifiant du commit>`.

Vous pouvez créer une branche à partir d'un commit spécifique en utilisant la commande `git checkout -b <nom de la branche> <identifiant du commit>`. Cela vous permet de créer une nouvelle branche à partir d'un commit existant et de continuer à travailler à partir de là.

Tous droits réservés à STUDI - Reproduction interdite

Exemple **Git checkout pour restaurer un fichier à une version précédente**

La commande `git checkout <commit> -- <chemin vers le fichier>` permet de restaurer un fichier à une version précédente spécifique.

Lorsque vous utilisez git checkout pour restaurer un fichier, assurez-vous d'avoir sauvegardé les modifications non enregistrées, car cela écrasera les modifications actuelles avec la version précédente. Soyez prudent lors de l'utilisation de cette commande pour éviter toute perte de données.

Méthode **Git diff**

La commande git diff permet de comparer les différences entre deux commits, branches ou fichiers. La syntaxe de base est : `git diff <commit1> <commit2>`, où <commit1> et <commit2> sont les identifiants des commits à comparer.

Comparer les différences entre le dernier commit et l'avant-dernier commit :

```
1 git diff HEAD HEAD~1
```

Cette commande affiche les différences entre le dernier commit et l'avant-dernier commit dans l'historique.

Comparer les différences entre deux branches :

```
1 git diff branche1 branche2
```

Cette commande affiche les différences entre les commits des deux branches spécifiées.

Méthode **Rechercher des mots-clés dans l'historique avec git grep**

La commande git grep permet de rechercher des mots-clés spécifiques dans l'historique des commits. La syntaxe de base est la suivante : git grep <mot-clé>.

Rechercher un mot-clé dans tous les fichiers de l'historique :

```
1 git grep "mot-clé"
```

Cette commande affiche toutes les occurrences du mot-clé spécifié dans tous les fichiers de l'historique.

Rechercher un mot-clé dans un fichier spécifique de l'historique :

```
1 git grep "mot-clé" -- chemin/vers/fichier
```

En remplaçant chemin/vers/fichier par le chemin réel du fichier, cette commande recherche le mot-clé uniquement dans ce fichier spécifique de l'historique.

La commande git grep est utile pour trouver rapidement des informations spécifiques dans l'historique des commits, ce qui peut être particulièrement pratique lors de la recherche de modifications ou de références à des fonctionnalités spécifiques.

B. Exercice : Quiz

[solution n°2 p.14]

Question 1

La commande `git log --author="John Doe"` permet de :

- ○ Afficher tous les commits réalisés par l'auteur « *John Doe* »
- ○ Afficher les différences entre deux branches
- ○ Restaurer un fichier à une version précédente

Question 2

Tous droits réservés à STUDI - Reproduction interdite

La commande `git show abcdefg` affiche :

- Les différences entre deux commits
- Les détails d'un commit spécifique identifié par « *abcdefg* »
- Les modifications apportées par un commit

Question 3

La commande `git diff HEAD HEAD~1` permet de :

- Comparer les différences entre le dernier commit et l'avant-dernier commit
- Créer une nouvelle branche à partir d'un commit spécifique
- Rechercher un mot-clé dans tous les fichiers de l'historique

Question 4

La commande `git checkout <commit> -- <chemin vers le fichier>` permet de :

- Afficher les détails d'un commit spécifique
- Restaurer un fichier à une version précédente spécifique
- Créer une nouvelle branche à partir d'un commit spécifique

Question 5

La commande git grep « *mot-clé* » permet de :

- Comparer les différences entre deux commits
- Afficher les détails d'un commit spécifique
- Rechercher un mot-clé dans tous les fichiers de l'historique

IV. Essentiel

Comprendre l'utilisation de Git log et interagir avec sa sortie est essentiel pour naviguer efficacement dans l'historique des commits d'un projet Git. La commande Git log permet d'afficher une liste chronologique des commits avec des informations détaillées telles que les auteurs, les dates et les messages de commit. En utilisant différentes options, comme le filtrage par auteur, par période ou par branche, il est possible de cibler les commits spécifiques qui nous intéressent.

De plus, des commandes telles que Git show, Git checkout, Git diff et Git grep offrent des fonctionnalités supplémentaires pour afficher les détails d'un commit, restaurer des fichiers à des versions antérieures, comparer les différences entre les commits ou rechercher des mots-clés dans l'historique. Ces fonctionnalités offrent une grande flexibilité lors de l'exploration et de l'analyse de l'évolution d'un projet dans Git. En maîtrisant ces concepts et commandes, les développeurs peuvent tirer pleinement parti des fonctionnalités de Git pour gérer leur code de manière efficace.

V. Auto-évaluation

A. Exercice

Vous êtes responsable du développement d'un projet logiciel basé sur Git. Votre tâche actuelle consiste à analyser l'historique des commits d'un dépôt Git afin de répondre à certaines questions spécifiques.

Tous droits réservés à STUDI - Reproduction interdite

Question 1

[solution n°3 p.15]

Voici le retour du git log, et les questions auxquels vous devez répondre :

```
commit 98c09d7a532a0879c47bf8fbf3041fd2873d3e3c
Author: John Doe johndoe@example.com
Date: Mon May 10 14:23:17 2023 +0300

Added new feature XYZ

commit b7d28ef08eac9a6f54289bcdef87e53a621a6f8e
Author: Jane Smith janesmith@example.com
Date: Fri May 7 09:45:52 2023 -0400

Updated README file


commit a2b39f827930e13f54ac9ce9d3e3e3f1f8b50c28
Author: Mike Johnson mikejohnson@example.com
Date: Wed May 5 18:12:34 2023 +0200

Fixed bug in login functionality

commit c8e51d69f7a830d5a81cf7c8e66f40624b7d38bc
Author: Sarah Thompson sarahthompson@example.com
Date : Mon May 3 10:07:16 2023 -0700

Initial commit
```

1. Combien de commits sont répertoriés dans ce retour de « *git log* » ?
2. Quel est l'auteur du commit qui a ajouté la nouvelle fonctionnalité « *XYZ* » ?
3. Quelles sont la date et l'heure de création du commit qui a corrigé un bogue dans la fonctionnalité de connexion (login) ?
4. Quel est le message du commit initial ?

Question 2

[solution n°4 p.15]

Cette fois-ci, c'est le retour d'un git show que vous devez analyser. Le voici :

```
commit 98c09d7a532a0879c47bf8fbf3041fd2873d3e3c
Author: John Doe johndoe@example.com
Date: Mon May 10 14:23:17 2023 +0300

Added new feature XYZ
diff --git a/path/to/file.txt b/path/to/file.txt
index abcd123..efgh456 100644
--- a/path/to/file.txt
+++ b/path/to/file.txt
@@ -1,4 +1,4 @@
-Hello, world!
+Bonjour, monde!
```

Voici les questions que vous devez vous poser :

1. Quelle est la différence entre la version précédente et la version actuelle du fichier « *file.txt* » ?
2. Quelles sont la date et l'heure de création du commit ?
3. Quel est le message associé au commit ?
4. Qui est l'auteur du commit ?

Tous droits réservés à STUDI - Reproduction interdite

B. Test

Exercice 1 : Quiz

[solution n°5 p.15]

Question 1

Git sauvegarde uniquement les 1000 derniers commits, ceux avant sont supprimés pour avoir une meilleure gestion de la mémoire.

- O Vrai
- O Faux

Question 2

La commande « `git log` » affiche l'historique des commits dans l'ordre chronologique inverse.

- O Vrai
- O Faux

Question 3

La commande « `git show` » affiche les détails d'un commit spécifique, y compris les modifications apportées.

- O Vrai
- O Faux

Question 4

Quelle commande Git permet de revenir à un commit en particulier ?

- O git revert
- O git reset
- O git checkout

Question 5

Quelle commande Git permet de visualiser les modifications apportées à un fichier spécifique dans un commit donné ?

- O git diff
- O git status
- O git log

Question 6

Quelle commande permet de rechercher un motif dans le contenu des fichiers ?

- O grep
- O find
- O sed

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 6 Solution n°1

Question 1

Qu'est-ce qu'un commit dans Git ?

- [x] Un instantané des fichiers du projet à un moment précis
- [ ] Une version des fichiers du projet incluant les changements depuis le dernier commit

Un commit dans Git est une action qui enregistre une ou plusieurs modifications dans un dépôt Git. C'est une manière de créer un instantané des fichiers du projet à un moment précis, en enregistrant les changements qui ont été apportés depuis le dernier commit. Chaque commit est accompagné d'un message de commit pour expliquer les changements.

Question 2

Quelle commande permet d'afficher l'historique des commits dans un dépôt Git ?

- [x] git log
- [ ] git history
- [ ] git show

La commande « *git log* » permet d'afficher l'historique des commits dans un dépôt Git. En exécutant cette commande, vous obtenez une liste des commits effectués, triés par ordre chronologique. Chaque commit est affiché avec des informations telles que l'identifiant, l'auteur, la date et le message du commit.

Question 3

Quelles informations sont affichées pour chaque commit dans git log ?

- [x] Identifiant du commit, auteur, date du commit, message du commit
- [ ] Numéro de version, date du commit, auteur, message du commit
- [ ] Identifiant du commit, date du commit, message du commit, fichiers modifiés

Pour chaque commit affiché dans « *git log* », les informations suivantes sont affichées : l'identifiant du commit (une chaîne de caractères unique), l'auteur du commit, la date du commit et le message du commit. Ces informations aident à comprendre les changements effectués dans chaque commit.

Question 4

Comment quitter l'affichage de git log et revenir à l'invite de commande ?

- [ ] Appuyer sur la touche x
- [x] Appuyer sur la touche q
- [ ] Appuyer sur la touche Ctrl + C

Quitter l'affichage de « *git log* » et revenir à l'invite de commande, vous pouvez appuyer sur la touche q (pour « *quitter* ») de votre clavier. Cela annulera l'affichage de « *git log* » et vous ramènera à l'endroit où vous avez exécuté la commande.

Question 5

Comment afficher les x derniers commits dans « *git log* » ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ git log -n x
- ○ git log --last=x
- ◉ git log -x

Pour afficher les x derniers commits dans « *git log* », vous pouvez utiliser l'option -x, où x représente le nombre de commits à afficher. Par exemple, pour afficher les 3 derniers commits, vous pouvez exécuter la commande « *git log -3* ». Cette option vous permet de limiter l'affichage de l'historique aux commits les plus récents.

Exercice p. 8 **Solution n°2**

Question 1

La commande `git log --author="John Doe"` permet de :

- ◉ Afficher tous les commits réalisés par l'auteur « *John Doe* »
- ○ Afficher les différences entre deux branches
- ○ Restaurer un fichier à une version précédente

La commande `git log --author="John Doe"` permet d'afficher tous les commits réalisés par l'auteur « *John Doe* ». Vous pouvez aussi ajouter l'option -p pour afficher les détails de chaque commits.

Question 2

La commande `git show abcdefg` affiche :

- ○ Les différences entre deux commits
- ◉ Les détails d'un commit spécifique identifié par « *abcdefg* »
- ○ Les modifications apportées par un commit

La commande `git show abcdefg` affiche les détails d'un commit spécifique identifié par « *abcdefg* ».

Question 3

La commande `git diff HEAD HEAD~1` permet de :

- ◉ Comparer les différences entre le dernier commit et l'avant-dernier commit
- ○ Créer une nouvelle branche à partir d'un commit spécifique
- ○ Rechercher un mot-clé dans tous les fichiers de l'historique

La commande `git diff HEAD HEAD~1` permet de comparer les différences entre le dernier commit et l'avant-dernier commit.

Question 4

La commande `git checkout <commit> -- <chemin vers le fichier>` permet de :

- ○ Afficher les détails d'un commit spécifique
- ◉ Restaurer un fichier à une version précédente spécifique
- ○ Créer une nouvelle branche à partir d'un commit spécifique

Tous droits réservés à STUDI - Reproduction interdite

La commande `git checkout <commit> -- <chemin vers le fichier>` permet de restaurer un fichier à une version précédente spécifique.

Question 5

La commande git grep « *mot-clé* » permet de :

- ○ Comparer les différences entre deux commits
- ○ Afficher les détails d'un commit spécifique
- ◉ Rechercher un mot-clé dans tous les fichiers de l'historique

La commande git grep « *mot-clé* » permet de rechercher un mot-clé dans tous les fichiers de l'historique.

p. 10 Solution n°3

1. Dans le retour de « *git log* », nous identifions quatre commits distincts. Chaque commit est représenté par une entrée qui commence par « *commit* » suivi d'un code unique. Dans ce cas, nous avons les commits suivants :
   - Commit 98c09d7a532a0879c47bf8fbf3041fd2873d3e3c
   - Commit b7d28ef08eac9a6f54289bcdef87e53a621a6f8e
   - Commit a2b39f827930e13f54ac9ce9d3e3e3f1f8b50c28
   - Commit c8e51d69f7a830d5a81cf7c8e66f40624b7d38bc
2. L'auteur du commit qui a ajouté la nouvelle fonctionnalité « *XYZ* » est John Doe. Son nom complet est mentionné dans l'entrée du commit : « *Author: John Doe johndoe@example.com* ».
3. Le commit qui a corrigé le bogue dans la fonctionnalité de connexion a été créé le 5 mai 2023 à 18 h 12 min 34 s, heure locale (+0200). La date et l'heure sont fournies dans l'entrée du commit : « *Date : Wed May 5 18:12:34 2023 +0200* ».
4. Le message du commit initial est « *Initial commit* ». Le message d'un commit est généralement une description brève des modifications apportées lors de la création du commit. Dans ce cas, le message du commit initial est précisément « *Initial commit* ».

p. 10 Solution n°4

1. La différence entre la version précédente et la version actuelle du fichier « *file.txt* » est la modification de la première ligne du contenu. La ligne précédente était « *Hello, world!* » et elle a été modifiée pour devenir « *Bonjour, monde!* ».
2. Le commit a été créé le lundi 10 mai 2023 à 14 h 23 min 17 s, avec un décalage horaire de +0300. Cela est indiqué dans l'entrée du commit : « *Date : Mon May 10 14:23:17 2023 +0300* ».
3. Le message associé au commit est « *Added new feature XYZ* ». Ce message décrit la modification apportée dans ce commit spécifique.
4. L'auteur du commit est John Doe. Son nom complet est mentionné dans l'entrée du commit : « *Author: John Doe johndoe@example.com* ».

Exercice p. 11 Solution n°5

Tous droits réservés à STUDI - Reproduction interdite

Question 1

Git sauvegarde uniquement les 1000 derniers commits, ceux avant sont supprimés pour avoir une meilleure gestion de la mémoire.

- ○ Vrai
- ◉ Faux

Git sauvegarde absolument tous les commits, même les plus anciens, ou même si vous avez des centaines de milliers de commits.

Question 2

La commande « `git log` » affiche l'historique des commits dans l'ordre chronologique inverse.

- ◉ Vrai
- ○ Faux

La commande « `git log` » affiche l'historique des commits dans l'ordre chronologique inverse, c'est-à-dire du commit le plus récent au plus ancien.

Question 3

La commande « `git show` » affiche les détails d'un commit spécifique, y compris les modifications apportées.

- ◉ Vrai
- ○ Faux

La commande « `git show` » affiche les détails d'un commit spécifique, y compris les modifications apportées. Elle affiche également les informations sur l'auteur, la date et le message du commit.

Question 4

Quelle commande Git permet de revenir à un commit en particulier ?

- ○ git revert
- ○ git reset
- ◉ git checkout

La commande « `git checkout` » permet de revenir à un commit en particulier, c'est-à-dire que votre environnement de travail sera identique au moment où le commit a été fait.

Question 5

Quelle commande Git permet de visualiser les modifications apportées à un fichier spécifique dans un commit donné ?

- ◉ git diff
- ○ git status
- ○ git log

La commande « `git diff` » permet de visualiser les modifications apportées à un fichier spécifique entre deux commits ou entre l'index et le répertoire de travail. Elle affiche les lignes ajoutées, modifiées ou supprimées.

Tous droits réservés à STUDI - Reproduction interdite

Question 6

Quelle commande permet de rechercher un motif dans le contenu des fichiers ?

- ◉ grep
- ○ find
- ○ sed

La commande « `grep` » est utilisée pour rechercher un motif spécifié dans le contenu des fichiers. Elle peut être utilisée avec différentes options pour personnaliser la recherche, comme l'ignorance de la casse, l'affichage des numéros de ligne, la recherche récursive dans les répertoires, etc. Cette commande est très utile pour trouver des occurrences spécifiques dans de nombreux fichiers.

Tous droits réservés à STUDI - Reproduction interdite